Deep Learning & GenAI

End-Semester Open-Book Exam Prep

30 Curated Questions — Numericals + Theory

IIT Bombay · ePGD · May 2026

Covers: Mid-Sem solutions · End-Sem practice set · TA exam-focus deep dive

Section	Topics	Q#
A	Language Modelling: N-grams, Perplexity, Entropy	1–5
B	Neural Networks: Backprop, Activations, CNNs	6–11
C	RNNs, LSTMs, BPTT	12–15
D	Transformers: Attention, Masking, Parameters	16–21
E	Pre-training, Fine-tuning, PEFT / LoRA	22–25
F	RLHF: Bradley-Terry, PPO-CLIP, DPO	26–29
G	GenAI: Seq2Seq, Decoding, RAG	30

SECTION A — Language Modelling: N-grams, Perplexity, Entropy

Q1. Bigram MLE Probability

A corpus contains the following sentences (with sentence markers): I am Sam Sam I am I do not like eggs . Vocabulary = {I, am, Sam, do, not, like, eggs, </s>}, so |V| = 8.

Given: count(I am) = 2, count(I) = 3, |V| = 8

Find: Unsmoothed P(am | I) and Add-1 smoothed P(am | I)

Formula: Unsmoothed: P(w2 | w1) = count(w1 w2) / count(w1) Add-1 smoothed: P(w2 | w1) = (count(w1 w2) + 1) / (count(w1) + 1 x |V|)

Step 1: Unsmoothed: P(am | I) = 2 / 3 = 0.667

Step 2: Add-1 smoothed: P(am | I) = (2 + 1) / (3 + 1x8) = 3 / 11 = 0.273

Answer: Unsmoothed = 0.667, Add-1 smoothed = 0.273

Note: Smoothing pulls the probability down because it redistributes mass to unseen bigrams. Zero probability for unseen n-grams makes perplexity infinite — smoothing prevents this.

Q2. Cross-Entropy and Perplexity from Model Probabilities

A bigram model scores a 4-token test sequence. The probability assigned to each actual next token is:

Given: Token 1 (the): q = 1/2

Given: Token 2 (cat): q = 1/4

Given: Token 3 (sat): q = 1/8

Given: Token 4 (down): q = 1/8

Given: N = 4 tokens

Find: (a) Per-token cross-entropy H in bits. (b) Perplexity PPL.

Formula: H = -(1/N) x SUM [log2(q(wt))] (sum over all N tokens) PPL = 2^H

Step 1: log2(1/2) = -1, log2(1/4) = -2, log2(1/8) = -3, log2(1/8) = -3

Step 2: Sum of logs = (-1) + (-2) + (-3) + (-3) = -9

Step 3: H = -(1/4) x (-9) = 9/4 = 2.25 bits/token

Step 4: PPL = 2^2.25 = 4 x 2^0.25 = 4 x 1.189 = 4.757

Answer: H = 2.25 bits/token, PPL ≈ 4.76

Note: Sanity check: PPL = (1/2 x 1/4 x 1/8 x 1/8)^(-1/4) = (1/512)^(-1/4) = 512^(1/4) = 4.757 ✓

Q3. Comparing Two Models via Perplexity

A test set of 100 tokens gives total log2-probability = -650 under Model A and total log2-probability = -700 under Model B.

Given: N = 100 tokens

Given: Sum of log2-probs (Model A) = -650

Given: Sum of log2-probs (Model B) = -700

Find: Which model is better? Compute perplexity for both.

Formula: H = -(1/N) x (sum of log2 probs) PPL = 2^H

Step 1: H_A = -(-650)/100 = 6.50 bits/token

Step 2: H_B = -(-700)/100 = 7.00 bits/token

Step 3: PPL_A = 2^6.50 = 2^6 x 2^0.5 = 64 x 1.414 = 90.5

Step 4: PPL_B = 2^7.00 = 128

Answer: Model A is better (lower H and lower PPL). PPL_A ≈ 90.5, PPL_B = 128

Q4. Good-Turing Smoothing — Adjusted Counts

A corpus has N = 100 tokens. The counts-of-counts are: N1 = 10 (words seen once), N2 = 4 (words seen twice), N3 = 2 (words seen three times).

Given: N = 100, N1 = 10, N2 = 4, N3 = 2

Find: (a) Good-Turing adjusted count c* for a word seen once (c=1). (b) Total probability mass reserved for unseen events.

Formula: Adjusted count: c* = (c + 1) x N(c+1) / N(c) Unseen mass: P(unseen) = N1 / N

Step 1: c* for c=1: c* = (1+1) x N2 / N1 = 2 x 4 / 10 = 8/10 = 0.80

Step 2: P(unseen) = N1 / N = 10 / 100 = 0.10 (10% of mass goes to unseen words)

Answer: c* = 0.80, Unseen mass = 10%

Note: A once-seen word is discounted to behave as if seen 0.8 times. MLE gives unseen words probability 0 — Good-Turing fixes this.

Q5. Katz Backoff — Alpha Weight

Explain the algebraic expression for the Katz backoff weight alpha(w_prev) that ensures the conditional distribution sums to 1. Then state why the numerator and denominator take those forms.

Formula: alpha(w_prev) = [1 - SUM over seen bigrams of P*(wi | w_prev)] / [SUM over unseen bigrams of P_unigram(wi)] where P* is the Good-Turing discounted probability for observed bigrams.

Step 1: All probabilities must sum to 1 over the vocabulary.

Step 2: The observed bigrams already consume SUM P*(wi | w_prev) of that mass.

Step 3: Numerator = leftover mass = 1 - (what observed bigrams used).

Step 4: Denominator = total unigram mass over unseen words — we rescale to distribute the leftover.

Answer: alpha redistributes the discounted leftover probability to unseen bigrams proportionally.

SECTION B — Neural Networks: Backprop, Activations, CNNs

Q6. Forward & Backward Pass — Chain Rule (Core Backprop)

A single hidden-layer network (no bias for simplicity): x = 2.0, w1 = 3.0, w2 = -1.0, b = 1.0. Computations: z1 = w1*x + b, a1 = ReLU(z1), z2 = w2*a1, loss = (z2 - 5.0)^2.

Given: x=2.0, w1=3.0, w2=-1.0, b=1.0, target=5.0

Find: (a) Forward pass values: z1, a1, z2, loss. (b) Gradients: d_loss/d_w2, d_loss/d_w1, d_loss/d_b, d_loss/d_x.

Formula: Forward: z1 = w1*x + b, a1 = max(0, z1), z2 = w2*a1, loss = (z2 - target)^2 Backward (chain rule): d_loss/d_z2 = 2*(z2 - target) d_loss/d_w2 = d_loss/d_z2 * a1 d_loss/d_a1 = d_loss/d_z2 * w2 d_loss/d_z1 = d_loss/d_a1 * ReLU_deriv(z1) [ReLU_deriv = 1 if z1>0, else 0] d_loss/d_w1 = d_loss/d_z1 * x d_loss/d_b = d_loss/d_z1 d_loss/d_x = d_loss/d_z1 * w1

Step 1: z1 = 3.0*2.0 + 1.0 = 7.0

Step 2: a1 = ReLU(7.0) = 7.0

Step 3: z2 = -1.0 * 7.0 = -7.0

Step 4: loss = (-7.0 - 5.0)^2 = (-12.0)^2 = 144.0

Step 5: d_loss/d_z2 = 2*(-12.0) = -24.0

Step 6: d_loss/d_w2 = -24.0 * 7.0 = -168.0

Step 7: d_loss/d_a1 = -24.0 * (-1.0) = 24.0

Step 8: ReLU_deriv(z1=7) = 1 → d_loss/d_z1 = 24.0 * 1 = 24.0

Step 9: d_loss/d_w1 = 24.0 * 2.0 = 48.0

Step 10: d_loss/d_b = 24.0 | d_loss/d_x = 24.0 * 3.0 = 72.0

Answer: loss=144.0, w2.grad=-168.0, w1.grad=48.0, b.grad=24.0, x.grad=72.0

Note: If x = -5.0: z1 = -14.0, ReLU(-14) = 0, so ALL gradients become 0 — the dying ReLU problem.

Q7. Sigmoid Derivative and Softplus

The Softplus activation is SP(x) = ln(1 + e^x). The sigmoid is sigma(x) = 1/(1 + e^(-x)).

Find: (a) Show d/dx [SP(x)] = sigma(x). (b) Compute d2/dx2 [SP(x)] and express in terms of sigma(x). (c) Behaviour as x → +inf and x → -inf.

Formula: d/dx ln(u) = (1/u) * du/dx Derivative of sigmoid: d/dx sigma(x) = sigma(x) * (1 - sigma(x))

Step 1: Let u = 1 + e^x, then d/dx [ln(u)] = e^x / (1 + e^x)

Step 2: Divide top and bottom by e^x: = 1 / (1 + e^(-x)) = sigma(x) ✓

Step 3: d2/dx2 [SP(x)] = d/dx [sigma(x)] = sigma(x)*(1 - sigma(x)) = e^x / (1+e^x)^2

Step 4: As x → +inf: sigma → 1, so sigma*(1-sigma) → 0. As x → -inf: sigma → 0, so → 0.

Step 5: Maximum of second derivative is 0.25 at x = 0.

Answer: (a) d/dx SP(x) = sigma(x). (b) d2/dx2 = sigma*(1-sigma), max=0.25. (c) Vanishes at both extremes — stable curvature, no gradient explosion.

Q8. CNN Output Shape Calculation

An encoder network applies three Conv2D layers in sequence, each with kernel=3, stride=2, padding=1. Input image shape: (1, 3, 64, 64). Channel sizes: 3→32→64→128.

Given: Input: (1, 3, 64, 64), k=3, s=2, p=1 for all layers

Find: Output shape after each Conv2D layer.

Formula: Output size along one spatial dim: out = floor((n + 2p - k) / s) + 1

Step 1: Layer 1: out = floor((64 + 2 - 3) / 2) + 1 = floor(63/2) + 1 = 31+1 = 32 → (1, 32, 32, 32)

Step 2: Layer 2: out = floor((32 + 2 - 3) / 2) + 1 = floor(31/2) + 1 = 15+1 = 16 → (1, 64, 16, 16)

Step 3: Layer 3: out = floor((16 + 2 - 3) / 2) + 1 = floor(15/2) + 1 = 7+1 = 8 → (1, 128, 8, 8)

Answer: Final output shape: (1, 128, 8, 8)

Q9. CNN Parameter Count and Linear Layer Bug

A student builds a CNN for 28x28 images: Conv2d(1,32, kernel=5, stride=1, no_pad) → MaxPool(2,2) → Conv2d(32,64, kernel=5, stride=1, no_pad) → MaxPool(2,2) → Linear(64*7*7, 10).

Given: Input: 28x28, no padding, kernel=5, stride=1, pool stride=2

Find: (a) Trace dimensions after each layer. (b) Will Linear(64*7*7, 10) crash? (c) Parameters in the Linear layer if corrected.

Formula: Conv (no pad): out = n - k + 1 MaxPool stride 2: out = floor(n / 2) Linear params = (in_features x out_features) + out_features (weights + biases)

Step 1: Conv1 (k=5, no pad): 28 - 5 + 1 = 24 → (B, 32, 24, 24)

Step 2: MaxPool(2,2): 24/2 = 12 → (B, 32, 12, 12)

Step 3: Conv2 (k=5, no pad): 12 - 5 + 1 = 8 → (B, 64, 8, 8)

Step 4: MaxPool(2,2): 8/2 = 4 → (B, 64, 4, 4) ← actual shape

Step 5: Correct flattened size = 64 x 4 x 4 = 1024 (code says 64*7*7 = 3136 — WRONG, will crash)

Step 6: Corrected Linear(1024, 10): params = 1024 x 10 + 10 = 10,250

Answer: Actual shape = (B, 64, 4, 4). Code WILL crash. Correct size = 1024. Linear params = 10,250.

Q10. PyTorch Code — Forward Pass Shape and Element Count

A model: Conv2d(1,8, k=3, s=1, p=1) → ReLU → MaxPool(2,2) → Conv2d(8,16, k=3, s=1, p=1) → ReLU → MaxPool(2,2). Input: torch.randn(1,1,28,28).

Given: Input: (1,1,28,28). Conv padding=1 preserves spatial size. MaxPool halves spatial dims.

Find: Output shape and total number of elements (out.numel()).

Formula: Conv2d with padding = kernel//2 preserves spatial dimensions. MaxPool(2,2) halves each spatial dimension. Total elements = product of all shape dimensions.

Step 1: Conv(1→8, pad=1): spatial preserved → (1, 8, 28, 28)

Step 2: MaxPool(2,2): 28/2=14 → (1, 8, 14, 14)

Step 3: Conv(8→16, pad=1): preserved → (1, 16, 14, 14)

Step 4: MaxPool(2,2): 14/2=7 → (1, 16, 7, 7)

Step 5: Total elements = 1 x 16 x 7 x 7 = 784

Answer: Shape: torch.Size([1, 16, 7, 7]) Total elements: 784

Q11. MSE Loss Forward Pass — Two-Layer Network (No Activation)

A two-layer network (no bias, no activation): W1 = [[1,2,3],[4,5,6]] (2x3), W2 = [[1,1]] (1x2). Input x = [1,1,1]^T, target y = 50.

Given: W1 shape 2x3, W2 shape 1x2, x=[1,1,1], y=50

Find: MSE loss = (y_hat - y)^2

Formula: h = W1 * x (matrix-vector product) y_hat = W2 * h MSE = (y_hat - y)^2

Step 1: h = W1*x: row1 = 1+2+3 = 6, row2 = 4+5+6 = 15 → h = [6, 15]^T

Step 2: y_hat = W2*h = 1*6 + 1*15 = 21

Step 3: MSE = (21 - 50)^2 = (-29)^2 = 841

Answer: MSE loss = 841

SECTION C — RNNs, LSTMs, and BPTT

Q12. RNN / GRU / LSTM Parameter Counts

A recurrent layer has input embedding size m = 100 and hidden size n = 128.

Given: m = 100 (input size), n = 128 (hidden size)

Find: Number of trainable parameters for one layer of: (a) Vanilla RNN (b) GRU (c) LSTM

Formula: One gate = affine map from [h_prev; x_t] of size (n+m) to a vector of size n. Parameters per gate = n*(n+m) + n = n*(n + m + 1) RNN = 1 gate x n*(n+m+1) GRU = 3 gates x n*(n+m+1) LSTM = 4 gates x n*(n+m+1)

Step 1: Base block = n*(m+n+1) = 128*(100+128+1) = 128*229 = 29,312

Step 2: RNN = 1 x 29,312 = 29,312

Step 3: GRU = 3 x 29,312 = 87,936

Step 4: LSTM = 4 x 29,312 = 117,248

Answer: RNN=29,312 GRU=87,936 LSTM=117,248

Note: GRU has 3 gates (reset r, update z, candidate h_tilde). LSTM has 4 gates (forget f, input i, candidate c_tilde, output o).

Q13. BPTT — Vanishing Gradient in Vanilla RNN

In a vanilla RNN: h_t = tanh(W * h_{t-1} + U * x_t). We want to backpropagate the gradient from final step T to step 1.

Find: (a) Write the expression for d_h_T / d_h_1 as a product of Jacobians. (b) Explain mathematically why gradients vanish for large T.

Formula: Chain rule over time: d_hT/d_h1 = PRODUCT from t=2 to T of (d_ht / d_h_{t-1}) Each factor: d_ht/d_h_{t-1} = diag(1 - tanh^2(z_t)) * W where z_t = W*h_{t-1} + U*x_t

Step 1: Each factor has magnitude at most ||W|| because tanh'(z) = 1 - tanh^2(z) is in [0, 1].

Step 2: ||d_hT/d_h1|| <= ||W||^(T-1)

Step 3: If ||W|| < 1: ||W||^(T-1) → 0 exponentially as T grows → VANISHING gradient.

Step 4: If ||W|| > 1: gradient can EXPLODE — fixed by gradient clipping.

Answer: Gradients decay as ||W||^(T-1). For ||W|| < 1 and large T, gradients → 0 and early time steps cannot be trained.

Q14. LSTM — Forget Gate and Long-Term Memory

An LSTM cell update: C_t = f_t * C_{t-1} + i_t * C_tilde_t. Information at t=1 must influence a prediction at t=100.

Find: (a) What value must f_t take for t=2 to 100 to preserve information? (b) How does this prevent vanishing gradients?

Formula: Gradient through cell state: d_Ct / d_C_{t-1} = f_t Gradient over many steps: d_C100 / d_C1 = PRODUCT from t=2 to 100 of f_t

Step 1: To preserve information: f_t ≈ 1 for all t = 2, …, 100.

Step 2: Then d_C100/d_C1 = PRODUCT of f_t ≈ 1^99 = 1.

Step 3: Gradient does NOT vanish — it remains ≈ 1 across 99 steps.

Step 4: Unlike RNN (multiplies W and tanh' repeatedly), the LSTM cell state has an ADDITIVE update path — a 'gradient highway' through time.

Answer: f_t ≈ 1 for t=2..100. Product of f_t ≈ 1 → gradient is preserved. This is the key advantage of LSTM over vanilla RNN.

Q15. RNN Sentiment Classification — Why Aggregate Hidden States

In sentiment classification with an RNN, two approaches exist: (i) use only the final hidden state h_T, or (ii) average all hidden states h_1 through h_T.

Find: Why might using only h_T fail, especially for long sequences?

Step 1: RNNs propagate information through h_T, but due to vanishing gradients, early hidden states (h_1, h_2, ...) are weakly encoded in h_T.

Step 2: A negative word at position 1 in a 50-word review may leave almost no trace in h_T.

Step 3: Averaging h_1..h_T retains contribution from ALL positions — earlier sentiment signals are not lost.

Answer: h_T suffers from vanishing gradients and 'forgets' early input. Aggregating all hidden states preserves the full sequence's information.

SECTION D — Transformers: Self-Attention, Masking, Parameter Counts

Q16. Scaled Dot-Product Attention — Manual Computation

Two tokens, d_k = 2. Matrices (rows = tokens): Q = [[1, 0], [0, 1]] K = [[1, 0], [1, 0]] V = [[1, 0], [0, 1]]

Given: q1 = [1,0], k1=[1,0], k2=[1,0], v1=[1,0], v2=[0,1], d_k=2

Find: Attention output for query row 1 (q1).

Formula: Step 1: score_j = dot(q, k_j) (dot product of query with each key) Step 2: scaled_j = score_j / sqrt(d_k) Step 3: alpha = softmax(scaled) (weights sum to 1) Step 4: output = SUM_j (alpha_j * v_j) (weighted sum of VALUES)

Step 1: score1 = [1,0].[1,0] = 1, score2 = [1,0].[1,0] = 1 → scores = [1, 1]

Step 2: scaled = [1/sqrt(2), 1/sqrt(2)] = [0.707, 0.707]

Step 3: Both scaled values equal → softmax gives alpha = [0.5, 0.5]

Step 4: output = 0.5*[1,0] + 0.5*[0,1] = [0.5, 0.5]

Answer: Attention output for q1 = [0.5, 0.5]

Note: When keys are identical the query splits attention evenly. The 1/sqrt(d_k) scaling prevents large dot products from saturating softmax.

Q17. Self-Attention — Two Output Rows

Q = [[1,1],[0,2]], K = [[1,0],[1,1]], V = [[2,0],[0,2]]. d_k = 2.

Given: q1=[1,1], q2=[0,2], k1=[1,0], k2=[1,1], v1=[2,0], v2=[0,2], sqrt(2)=1.414

Find: Compute both output rows of the attention.

Formula: For each query qi: scores = [dot(qi,k1), dot(qi,k2)] / sqrt(d_k) alpha = softmax(scores) output_i = alpha[0]*v1 + alpha[1]*v2

Step 1: Row 1 — q1=[1,1]: score1=1*1+1*0=1, score2=1*1+1*1=2

Step 2: scaled = [1/1.414, 2/1.414] = [0.707, 1.414]

Step 3: e^0.707=2.028, e^1.414=4.113, sum=6.141 → alpha=[0.330, 0.670]

Step 4: output1 = 0.330*[2,0] + 0.670*[0,2] = [0.660, 1.340]

Step 5: Row 2 — q2=[0,2]: score1=0+2=2, score2=0+2=2

Step 6: scaled = [1.414, 1.414] → alpha=[0.5, 0.5] → output2=0.5*[2,0]+0.5*[0,2]=[1,1]

Answer: Output row 1 ≈ [0.66, 1.34] Output row 2 = [1.0, 1.0]

Q18. Three Masking Types in Transformers

Describe the three distinct masking methods used in Transformer training and inference.

Step 1: CAUSAL MASK (decoder self-attention): Position l may not attend to any future position t>l. Scores at future positions are set to -infinity before softmax, so alpha → 0. This enforces autoregressive generation.

Step 2: PADDING MASK (encoder & decoder): Padding tokens added to short sequences must not influence attention. Their scores are set to -infinity so they contribute zero weight.

Step 3: MLM TOKEN MASK (BERT pre-training): 15% of tokens are selected. Of those: 80% replaced with [MASK], 10% with a random token, 10% left unchanged. The model predicts the original tokens at selected positions.

Answer: Three masks: Causal (autoregressive generation), Padding (batch handling), MLM token masking (BERT pre-training objective).

Q19. Transformer Encoder — Parameter Count

A transformer encoder: d_model = 512, layers L = 6, d_ff = 2048, vocab V = 30000, max_len = 512 (learned positional), output head TIED to input embedding.

Given: d=512, L=6, d_ff=2048, V=30000, n_pos=512, tied output head

Find: Total trainable parameters (weights only, ignoring biases).

Formula: Attention per layer: 4 * d^2 (W_Q, W_K, W_V, W_O each d x d) FFN per layer: 2 * d * d_ff (W1: d x d_ff, W2: d_ff x d) Per layer total: 4*d^2 + 2*d*d_ff All layers: L * (4*d^2 + 2*d*d_ff) Token embedding: V * d Positional (learned): n_pos * d Grand total: L*(4*d^2 + 2*d*d_ff) + V*d + n_pos*d

Step 1: 4*d^2 = 4 * 512^2 = 4 * 262,144 = 1,048,576 per layer (attention)

Step 2: 2*d*d_ff = 2 * 512 * 2048 = 2,097,152 per layer (FFN)

Step 3: Per layer = 1,048,576 + 2,097,152 = 3,145,728

Step 4: All 6 layers = 6 * 3,145,728 = 18,874,368

Step 5: Token embedding = 30000 * 512 = 15,360,000

Step 6: Positional embedding = 512 * 512 = 262,144 (sinusoidal PE = 0 params)

Step 7: Grand total = 18,874,368 + 15,360,000 + 262,144 = 34,496,512 ≈ 34.5M

Answer: ≈ 34.5 million parameters. If sinusoidal PE: 34.5M - 0.26M = 34.23M.

*Note: Dominant terms: FFN layers (~12.6M) and token embedding (~15.4M). Untied output head adds another V*d = 15.36M.*

Q20. BERT-Base Parameter Count (Full)

BERT-base configuration: d = 768, L = 12, d_ff = 3072, V = 30522, n_pos = 512 (learned). Output head TIED to embedding.

Given: d=768, L=12, d_ff=3072, V=30522, n_pos=512

Find: Approximate total parameters. Verify the 110M figure.

Formula: Same as Q19: L*(4*d^2 + 2*d*d_ff) + V*d + n_pos*d

Step 1: 4*d^2 = 4 * 768^2 = 4 * 589,824 = 2,359,296 (attention per layer)

Step 2: 2*d*d_ff = 2 * 768 * 3072 = 4,718,592 (FFN per layer)

Step 3: Per layer = 2,359,296 + 4,718,592 = 7,077,888

Step 4: 12 layers = 12 * 7,077,888 = 84,934,656

Step 5: Token embedding = 30522 * 768 = 23,440,896

Step 6: Positional embedding = 512 * 768 = 393,216

Step 7: Total ≈ 84,934,656 + 23,440,896 + 393,216 = 108,768,768 ≈ 110M ✓

Answer: ≈ 110M parameters — matches the known BERT-base figure.

Q21. Why Scaled Dot-Product? — Softmax Saturation

Explain why the dot products in self-attention are divided by sqrt(d_k) before softmax. What happens if this scaling is omitted for large d_k?

Step 1: If Q and K components have zero mean and unit variance, each element of the dot product q.k has variance 1.

Step 2: The dot product q.k is a sum of d_k such terms, so its total variance = d_k, and its standard deviation = sqrt(d_k).

Step 3: For large d_k (e.g. 512), dot products can be very large in magnitude.

Step 4: Large inputs push softmax into saturation (output ≈ one-hot), causing near-zero gradients and training stalls.

Step 5: Dividing by sqrt(d_k) restores the dot product to unit variance before softmax.

Answer: Without scaling, softmax saturates for large d_k → vanishing gradients. Dividing by sqrt(d_k) keeps gradients flowing.

SECTION E — Pre-training, Fine-tuning, and PEFT / LoRA

Q22. Last-Layer vs Full Fine-Tuning — Parameter Ratio

BERT-base has ≈ 110M parameters with hidden size d = 768. You fine-tune it for a 3-class classification task.

Given: Total model params = 110,000,000, d = 768, C = 3 classes

Find: (a) Parameters trained in last-layer fine-tuning. (b) Parameters trained in full fine-tuning. (c) Ratio. (d) Memory implication.

Formula: Linear head parameters = d * C + C = (d + 1) * C Ratio = head_params / total_params

Step 1: Head params = (768 + 1) * 3 = 769 * 3 = 2,307

Step 2: Full FT trains all 110,000,000 parameters

Step 3: Ratio = 2,307 / 110,000,000 ≈ 0.0021% (about 2.1 × 10^-5)

Step 4: Adam stores 3 values per trainable param (grad + 2 moments): Full FT ≈ 110M*3*4 bytes ≈ 1.3 GB. Last-layer ≈ 2307*3*4 ≈ 28 KB.

Answer: (a) 2,307 (b) 110M (c) 0.002% (d) 1.3 GB vs 28 KB optimizer state

Q23. LoRA — Parameter Savings

Apply LoRA with rank r = 8 to a single weight matrix W of shape 768 x 768. Then apply LoRA (r=8) to W_Q and W_V in all 12 layers of BERT-base.

Given: d = 768, r = 8, 2 matrices per layer (W_Q and W_V), 12 layers

Find: (a) LoRA params per matrix vs full matrix. (b) Percentage of full. (c) Total LoRA params across the model.

Formula: LoRA freezes W and learns low-rank update Delta_W = B * A where B has shape d x r and A has shape r x d. LoRA params per matrix = d*r + r*d = 2*d*r Full params per matrix = d*d = d^2

Step 1: LoRA per matrix = 2 * 768 * 8 = 12,288

Step 2: Full matrix = 768^2 = 589,824

Step 3: Percentage = 12,288 / 589,824 = 2.08%

Step 4: Total LoRA = 12,288 * 2 matrices * 12 layers = 294,912 ≈ 0.29M

Answer: (a) 12,288 vs 589,824 (b) 2.08% (c) 0.29M total LoRA params (0.27% of BERT-base)

Note: 0.27% of parameters updated, yet LoRA usually recovers most of full fine-tuning accuracy.

Q24. Prompt Tuning — Trainable Parameter Count

In soft prompt tuning, m learnable embedding vectors each of dimension d are prepended to the input. The rest of the model is frozen. GPT-3 has 175B parameters.

Given: m = 50 prompt tokens, d = 768, GPT-3 total = 175,000,000,000

Find: (a) Trainable parameters with prompt tuning. (b) Ratio vs GPT-3 full fine-tune.

Formula: Trainable params = m * d

Step 1: Prompt tuning params = 50 * 768 = 38,400

Step 2: Ratio = 38,400 / 175,000,000,000 ≈ 2.2 × 10^-7 (roughly 4.5 million times fewer)

Answer: 38,400 trainable parameters. Ratio ≈ 0.000022% of GPT-3.

Q25. MLM Masking Budget — BERT Pre-training

A 200-token sequence is pre-trained with BERT-style Masked Language Modeling.

Given: Sequence length = 200 tokens, BERT masking: select 15%, then split 80/10/10

Find: How many tokens become [MASK], random, and unchanged?

Formula: Selected = 0.15 * total_tokens Of selected: [MASK] = 80%, random = 10%, unchanged = 10%

Step 1: Selected = 0.15 * 200 = 30 tokens

Step 2: [MASK] tokens = 0.80 * 30 = 24

Step 3: Random tokens = 0.10 * 30 = 3

Step 4: Unchanged tokens = 0.10 * 30 = 3 (check: 24+3+3 = 30 ✓)

Answer: 24 [MASK], 3 random, 3 unchanged

Note: The 10% random + 10% unchanged prevent the model relying on [MASK] at inference. [MASK] never appears during fine-tuning — this mismatch is reduced by the 80/10/10 split.

SECTION F — RLHF: Bradley-Terry, PPO-CLIP, and DPO

Q26. Bradley-Terry Loss Derivation and Numerical Computation

The Bradley-Terry model gives: P(y_w beats y_l) = sigma(r_w - r_l). The reward model is trained by minimising the negative log-likelihood. Compute the loss for the following three preference pairs: Pair 1: r_w=2.0, r_l=1.0 | Pair 2: r_w=0.5, r_l=1.5 | Pair 3: r_w=3.0, r_l=0.0

Given: sigma(z) = 1/(1+e^-z). Three pairs of (r_w, r_l) as above.

Find: Per-pair loss and average loss across the three pairs.

Formula: Loss for one pair: L_i = -log(sigma(r_w - r_l)) Average loss: L = (1/3) * SUM of L_i sigma(z) = 1 / (1 + e^(-z))

Step 1: Pair 1: z=2.0-1.0=1.0, sigma(1.0)=1/(1+e^-1)=1/(1+0.368)=0.731, L1=-log(0.731)=0.313

Step 2: Pair 2: z=0.5-1.5=-1.0, sigma(-1.0)=1/(1+e^1)=1/(1+2.718)=0.269, L2=-log(0.269)=1.313

Step 3: Pair 3: z=3.0-0.0=3.0, sigma(3.0)=1/(1+e^-3)=1/(1+0.050)=0.953, L3=-log(0.953)=0.049

Step 4: Average loss = (0.313 + 1.313 + 0.049) / 3 = 1.675 / 3 = 0.558

Answer: L_avg ≈ 0.558. Pair 2 contributes most (1.313) — reward model ranked the REJECTED response higher.

Q27. Regularized RLHF Objective — KL Penalty

The RLHF objective penalises the policy for drifting from the SFT reference model: J(theta) = E[r(x,y) - beta * log(pi_theta(y|x) / pi_ref(y|x))] For one response: r(x,y) = 1.5, log(pi_theta / pi_ref) = 0.3, beta = 0.1.

Given: r = 1.5, log_ratio = 0.3, beta = 0.1

Find: (a) Regularized reward r_s. (b) Interpret the result. (c) What happens if log_ratio = 15?

Formula: Regularized reward: r_s(x,y) = r(x,y) - beta * log(pi_theta(y|x) / pi_ref(y|x))

Step 1: r_s = 1.5 - 0.1 * 0.3 = 1.5 - 0.03 = 1.47

Step 2: Small KL (0.03): policy barely drifted from SFT. Response is beneficial — keep it.

Step 3: If log_ratio=15: r_s = 1.5 - 0.1*15 = 1.5 - 1.5 = 0.0 → borderline reward-hacking.

Step 4: If log_ratio=25: r_s = 1.5 - 2.5 = -1.0 → suppress this response.

Answer: r_s = 1.47 (beneficial, low KL). Large log_ratio signals reward hacking and yields negative r_s.

Q28. PPO-CLIP Objective — Three Cases

PPO-CLIP objective with epsilon = 0.2: L_CLIP = min(rho * A_hat, clip(rho, 1-eps, 1+eps) * A_hat) Case (a): A_hat = +2, rho = 1.5. Case (b): A_hat = +2, rho = 0.7. Case (c): A_hat = -2, rho = 1.5.

Given: eps=0.2, clip range=[0.8, 1.2]. Three (A_hat, rho) cases above.

Find: L_CLIP value for each case.

Formula: term1 = rho * A_hat clipped_rho = min(max(rho, 1-eps), 1+eps) term2 = clipped_rho * A_hat L_CLIP = min(term1, term2)

Step 1: Case (a): A=+2, rho=1.5. term1=1.5*2=3.0. clip(1.5→1.2): term2=1.2*2=2.4. min(3.0,2.4)=2.4

Step 2: Case (b): A=+2, rho=0.7. term1=0.7*2=1.4. clip(0.7→0.8): term2=0.8*2=1.6. min(1.4,1.6)=1.4

Step 3: Case (c): A=-2, rho=1.5. term1=1.5*(-2)=-3.0. clip(1.5→1.2): term2=1.2*(-2)=-2.4. min(-3.0,-2.4)=-3.0

Answer: (a) L_CLIP = 2.4 (b) L_CLIP = 1.4 (c) L_CLIP = -3.0

Note: Clipping removes the incentive to over-optimise on good actions, but still penalises taking bad actions more likely. Each update stays conservative.

Q29. DPO — Direct Preference Optimisation

DPO eliminates the separate reward model. It uses the identity that the RLHF optimal policy is pi*(y|x) proportional to pi_ref(y|x) * exp(r(y)/beta) to write the reward in terms of log-prob ratios.

Find: (a) Write the DPO loss. (b) Explain why no reward model or RL loop is needed.

Formula: L_DPO = -log sigma(beta * [log(pi_theta(y_w|x)/pi_ref(y_w|x)) - log(pi_theta(y_l|x)/pi_ref(y_l|x))])

Step 1: From the optimal-policy identity, the reward of response y can be written as: r(y) = beta * log(pi*(y|x)/pi_ref(y|x)) + beta*log Z(x)

Step 2: Substituting into Bradley-Terry loss, the partition function Z(x) cancels.

Step 3: The resulting loss depends only on log-prob ratios under pi_theta and pi_ref — both of which are available without a reward model.

Step 4: Training: simply raise the log-prob of y_w relative to pi_ref, while lowering it for y_l. No PPO, no reward model, no RL rollouts.

Answer: DPO re-parametrises the reward via the policy itself, collapsing RLHF to a single supervised-style loss over preference pairs.

SECTION G — Seq2Seq, Decoding Strategies, and RAG

Q30. Greedy vs Beam Search — Two-Step Decoding

Vocabulary = {A, B}, two decoding steps. Step 1 priors: P(A)=0.6, P(B)=0.4. Continuations from A: P(A|A)=0.5, P(B|A)=0.5. Continuations from B: P(A|B)=0.95, P(B|B)=0.05.

Given: P(A)=0.6, P(B)=0.4, P(A|A)=0.5, P(B|A)=0.5, P(A|B)=0.95, P(B|B)=0.05

Find: (a) Greedy output and its probability. (b) Beam-2 best sequence and probability. (c) Why greedy is suboptimal here.

Formula: Sequence probability = P(token1) * P(token2 | token1) Greedy: at each step pick argmax of current distribution. Beam (width 2): enumerate all length-2 paths, pick highest.

Step 1: Greedy step 1: max(0.6, 0.4) → pick A. Step 2: max(0.5, 0.5) → pick A (or B, tie).

Step 2: Greedy path AA: P = 0.6 * 0.5 = 0.30

Step 3: All beam paths:

Step 4: AA = 0.6 * 0.5 = 0.30

Step 5: AB = 0.6 * 0.5 = 0.30

Step 6: BA = 0.4 * 0.95 = 0.38 ← HIGHEST

Step 7: BB = 0.4 * 0.05 = 0.02

Step 8: Beam-2 selects BA with probability 0.38.

Answer: Greedy → AA (P=0.30). Beam-2 → BA (P=0.38).

Note: Greedy committed to A (best at step 1), but B leads to a much better continuation (0.95). Greedy's locally-best choice was globally suboptimal. Beam search finds the higher-probability path.

QUICK REFERENCE — Key Formulas at a Glance

Concept	Formula
Bigram MLE	P(w2|w1) = count(w1 w2) / count(w1)
Add-1 smoothed bigram	P(w2|w1) = (count(w1 w2)+1) / (count(w1) + |V|)
Cross-entropy (bits)	H = -(1/N) * SUM log2 q(wt)
Perplexity	PPL = 2^H or PPL = exp(H)
Good-Turing c*	c* = (c+1) * N(c+1) / N(c)
Scaled dot-product	Attention = softmax(Q*K^T / sqrt(d_k)) * V
Conv output size	out = floor((n + 2p - k) / s) + 1
RNN params	n*(m + n + 1)
LSTM params	4 * n*(m + n + 1)
GRU params	3 * n*(m + n + 1)
Transformer per layer	4*d^2 + 2*d*d_ff
LoRA params	2 * d * r per weight matrix
BT loss (one pair)	L = -log sigma(r_w - r_l)
RLHF reg. reward	r_s = r - beta * log(pi_theta / pi_ref)
PPO-CLIP	min(rho*A, clip(rho, 1-eps, 1+eps)*A)
DPO loss	-log sigma(beta * [log ratio_w - log ratio_l])
MLM masking	15% selected: 80% [MASK], 10% random, 10% unchanged
Prompt tuning params	m * d

Good luck on your exam! All questions sourced from mid-sem solutions, end-sem practice set, and TA exam-focus deep dive.